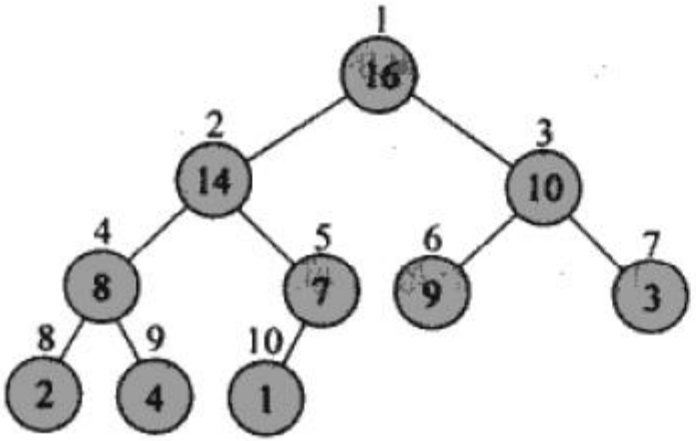
第6章　堆排序

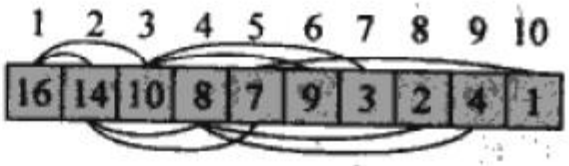
6.1 堆

- (二叉)堆数据结构是一种数组对象，它可以被视为一颗完全二叉树，树中每个节点和数组中存放该节点值的那个元素对应。如果表示堆的数组为A，那么树的根为A[1]。
- 表示堆的数组A是一个具有两个属性的对象：length(A)是数组中的元素个数，heap-size(A)是存放在A中的堆的元素个数；A[heap-size(A)]之后的元素都不属于相应的堆。也就是：Heap-size(A)<=length(A)
- 给定某个节点的下标i，其父节点PARENT(i)，左儿子LEFT(i)和右儿子RIGHT(i)可以表示为
  - PARENT(i): $\lfloor i / 2 \rfloor$
  - LEFT(i): 2i
  - RIGHT(i): 2i + 1

堆的示例



a)



b)

一个最大堆(大根堆)可被看作 a)一棵二叉树和b)一个数组。圆圈中的数字表示树中每个结点存储的值，结点上方的数字表示对应的数组下标。数组上下的连线表示父子关系，且父结点总在子结点的左边。这棵树的高度为 3，存储值为 8 的 4 号结点的高度为 1

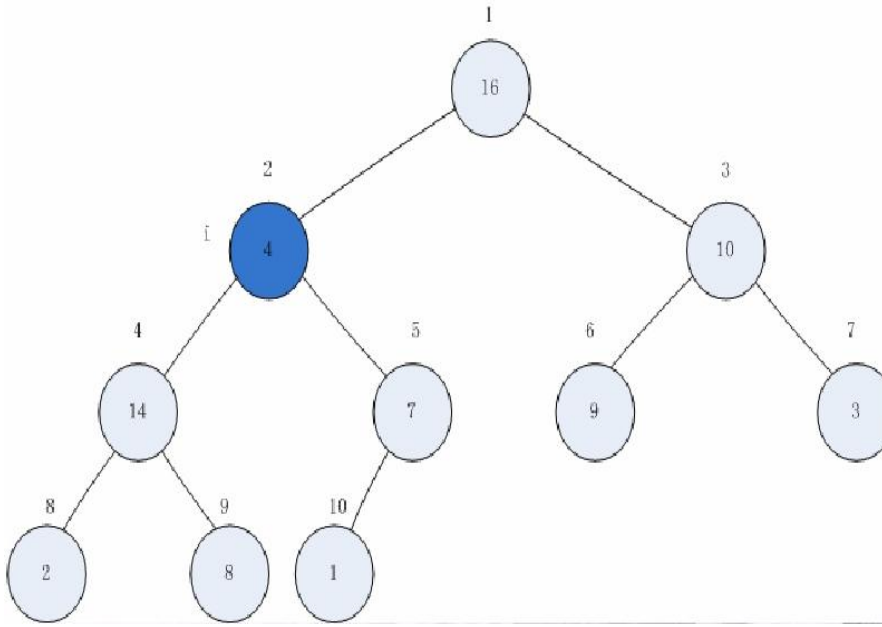
6.1 最大堆和最小堆

- 二叉堆有两种：最大堆和最小堆
  - 对于最大堆，除了根以外的每个节点，有
    - $A[PARENT(i)] \geq A[i]$
    - 这样，堆中的最大元素就存放在根节点中，并且在以某一个节点为根的子树中，各节点的值都不大于该子树根节点的值
  - 对于最小堆，除了根以外的每个节点，有
    - $A[PARENT(i)] \leq A[i]$，最小堆的最小元素在跟部。
- 堆可以被看成一棵树，节点在堆中的高度定义为从本节点到叶子的最长简单下降路径上边的个数。堆的高度定义为树根的高度，为$\Theta(\lg n)$。

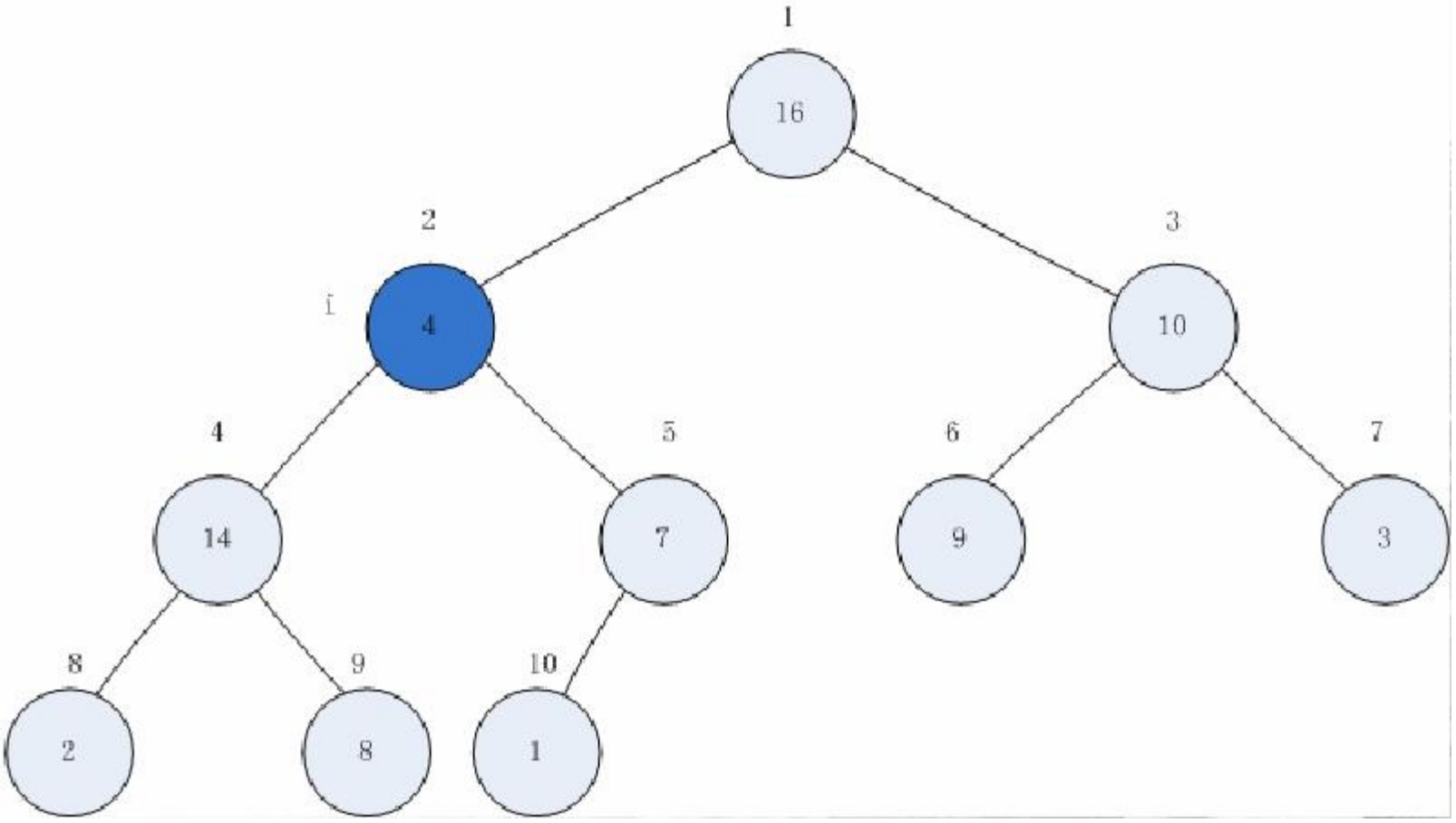
6.2 保持堆的性质

- MAX-HEAPIFY是对最大堆进行操作的重要的子程序，调整A[i]在堆中的位置，使以i为根的子树为最大堆

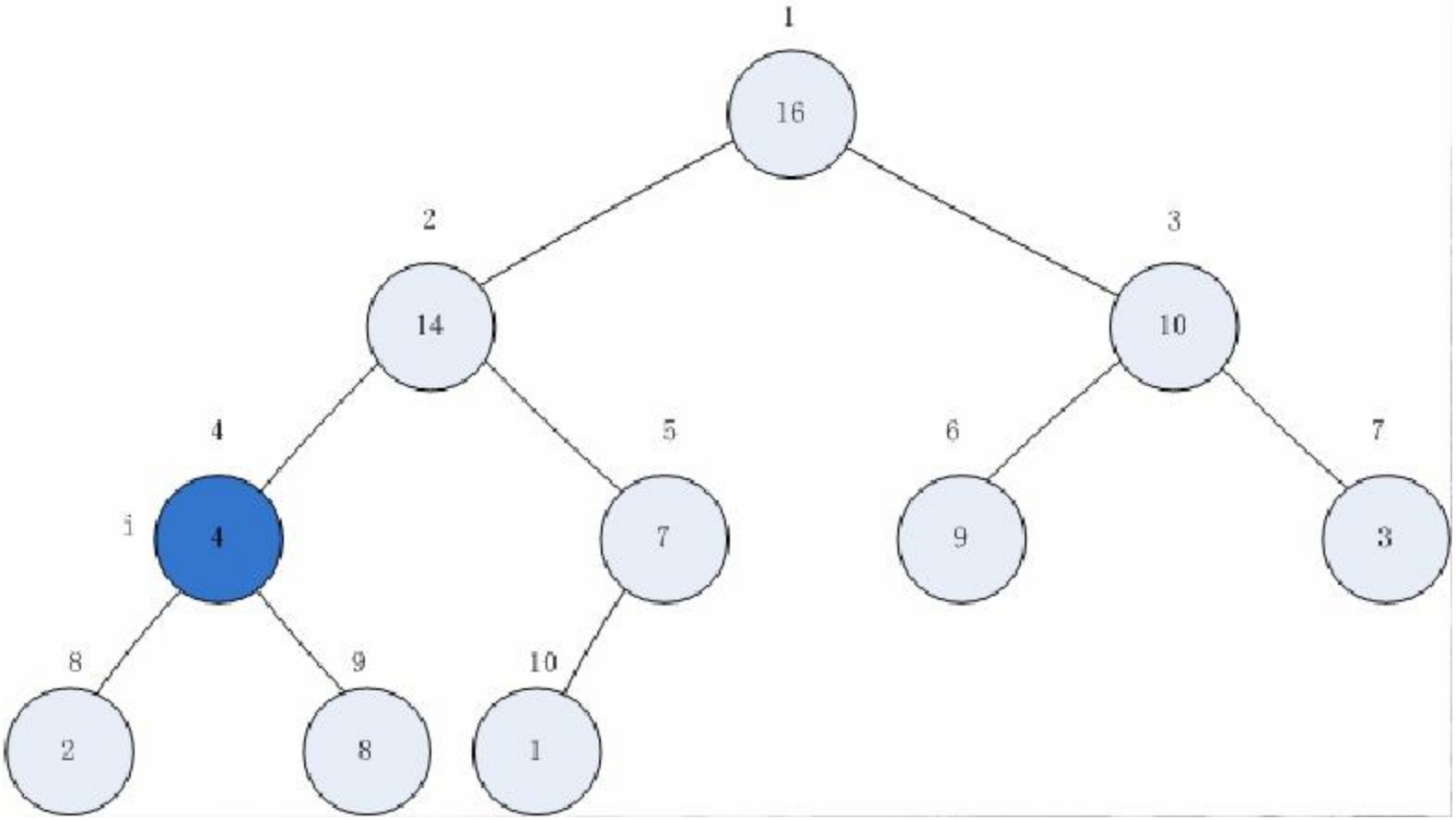
```
MAX-HEAPIFY(A,i)
1  l ← LEFT(i)
2  r ← RIGHT(i)
3  If l ≤ heap-size[A] and A[l] > A[i]
4    then largest ← l
5    else largest ← i
6  If r ≤ heap-size[A] and A[r] > A[largest]
7    then largest ← r
8  If largest ≠ i
9      then exchange A[i]↔A[largest]
10             MAX-HEAPIFY(A, largest)
```



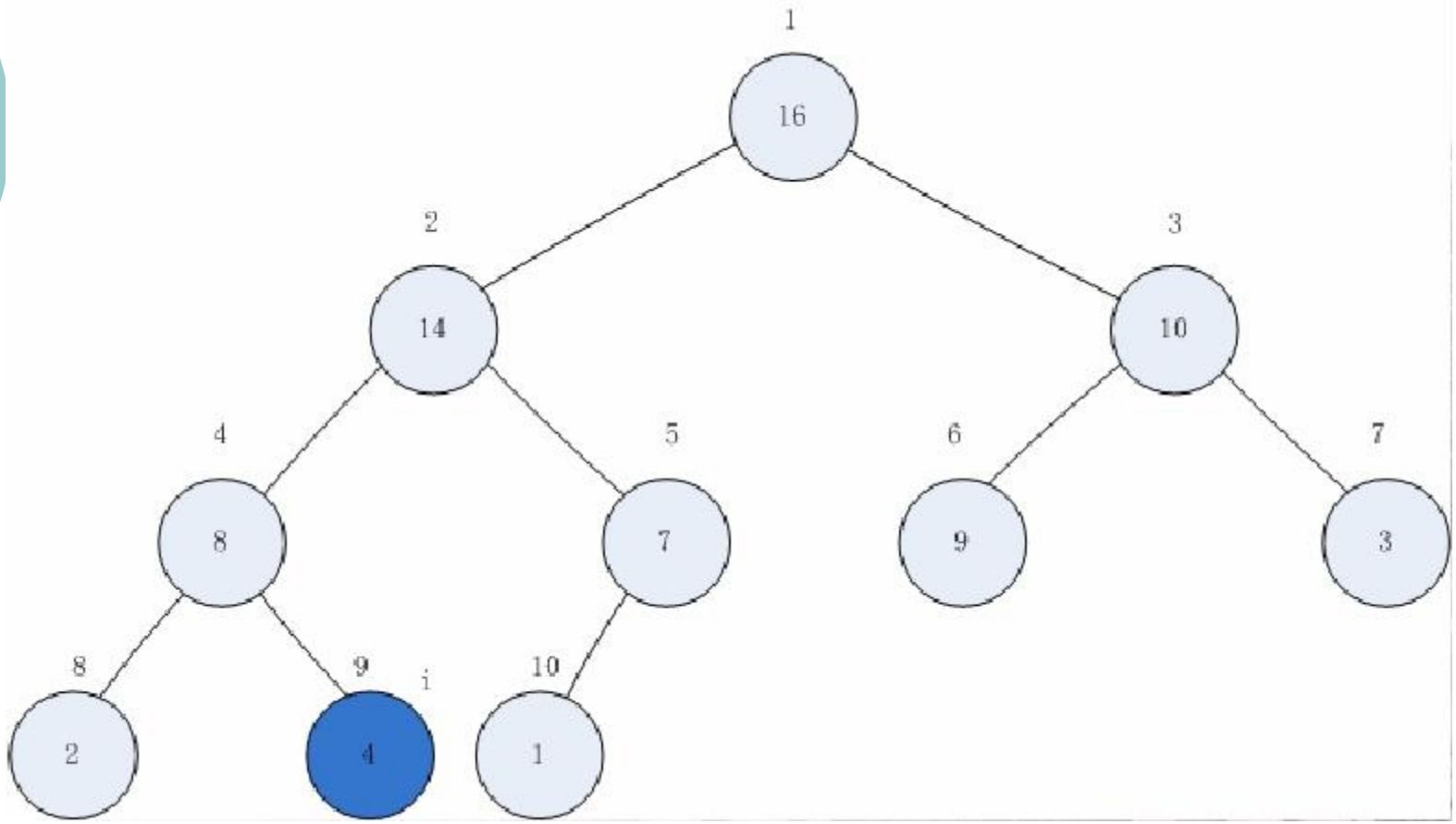
6.1 MAX-HEAPIFY过程



6.1 MAX-HEAPIFY过程



6.1 MAX-HEAPIFY过程



6.2 MAX-HEAPIFY时间

- 当MAX-HEAPIFY作用在一棵以节点i为根的，大小为n的子树上时，其运行时间为调整元素A[i]，A[LEFT(i)]和A[RIGHT(i)]的关系时所用时间$\Theta(1)$，再加上对以i的某个子结点为根的子树递归调用MAX-HEAPIFY所需的时间。i结点的子树大小至多为2n/3，那么MAX-HEAPFY的运行时间为：

$$T(n) \leq T(2n/3) + \Theta(1)$$

根据主定理，该递归式的解为$T(n) \leq \Theta(\lg n)$

课内练习6.1

1、采用分治法设计一个递归算法，构建最大堆。
2、分析构建最大堆递归算法时间复杂度，列出时间复杂度递归式，并用主方法求解。

6.3 建堆

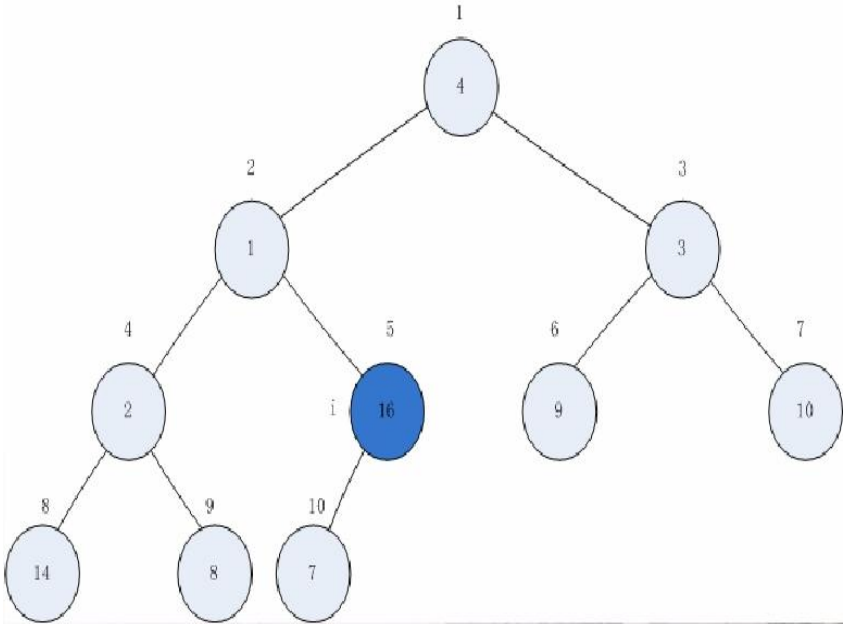
- 我们可以自底向上的用MAX-HEAPIFY来将一个数组A[1..n]变成一个最大堆。过程BUILD-MAX-HEAP对树中的每一个其他节点都调用一次MAX-HEAPIFY。

BUILD-MAX-HEAP(A)

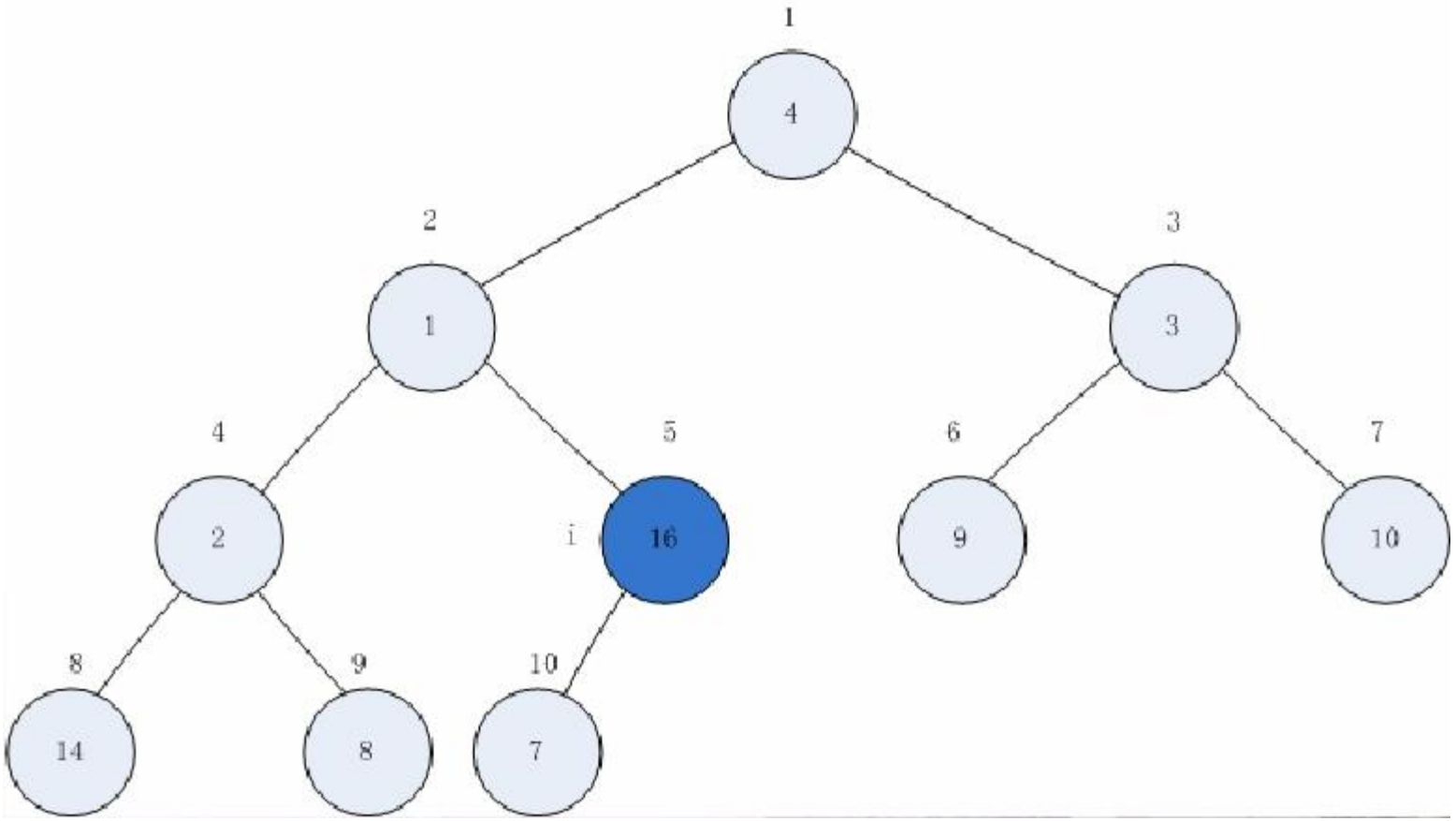
1 heap-size[A] ← length[A]

2 for i ← ⌊length[*A*] / 2⌋ downto 1

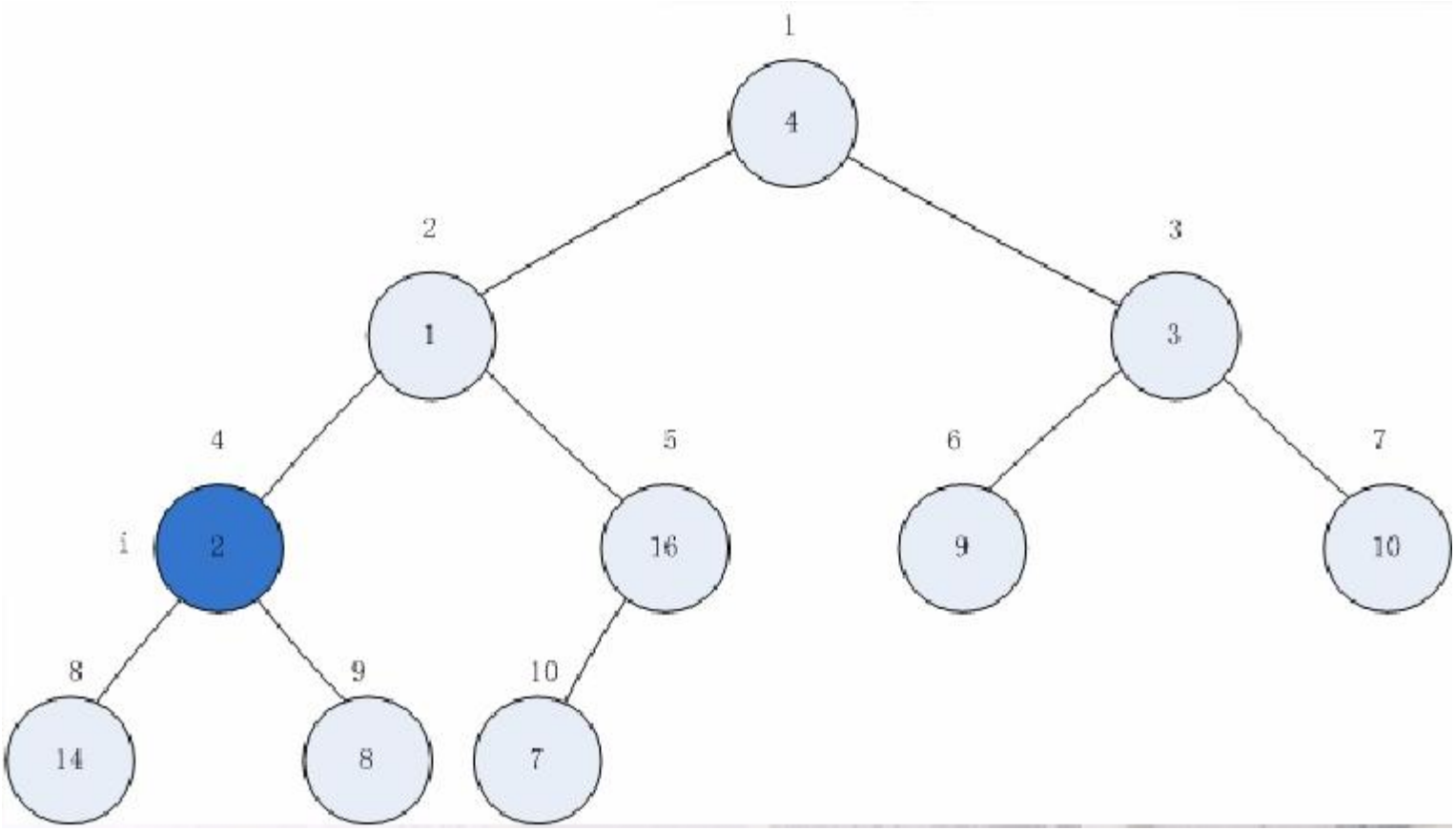
do MAX-HEAPIFY(A,i)



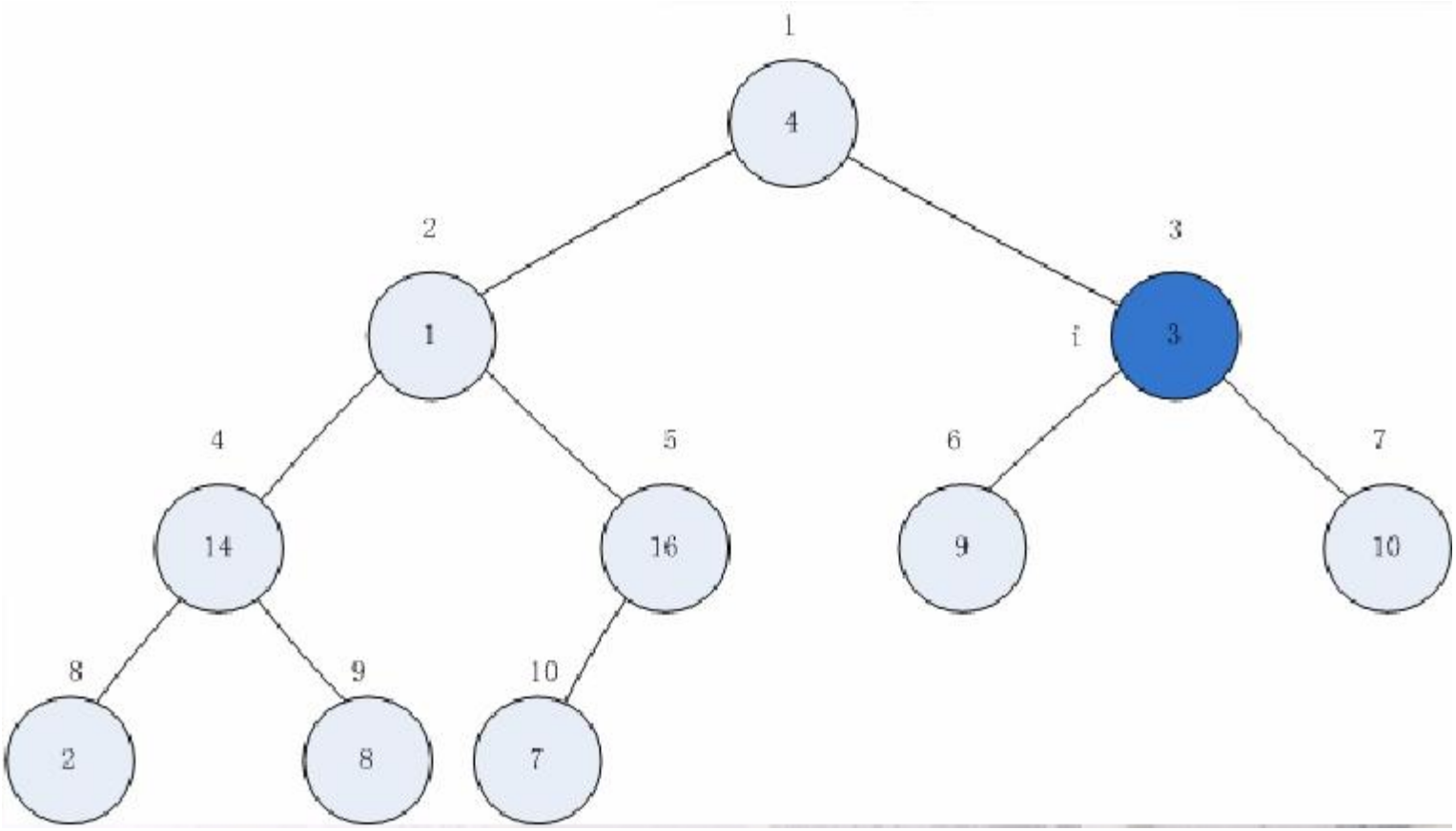
建堆过程



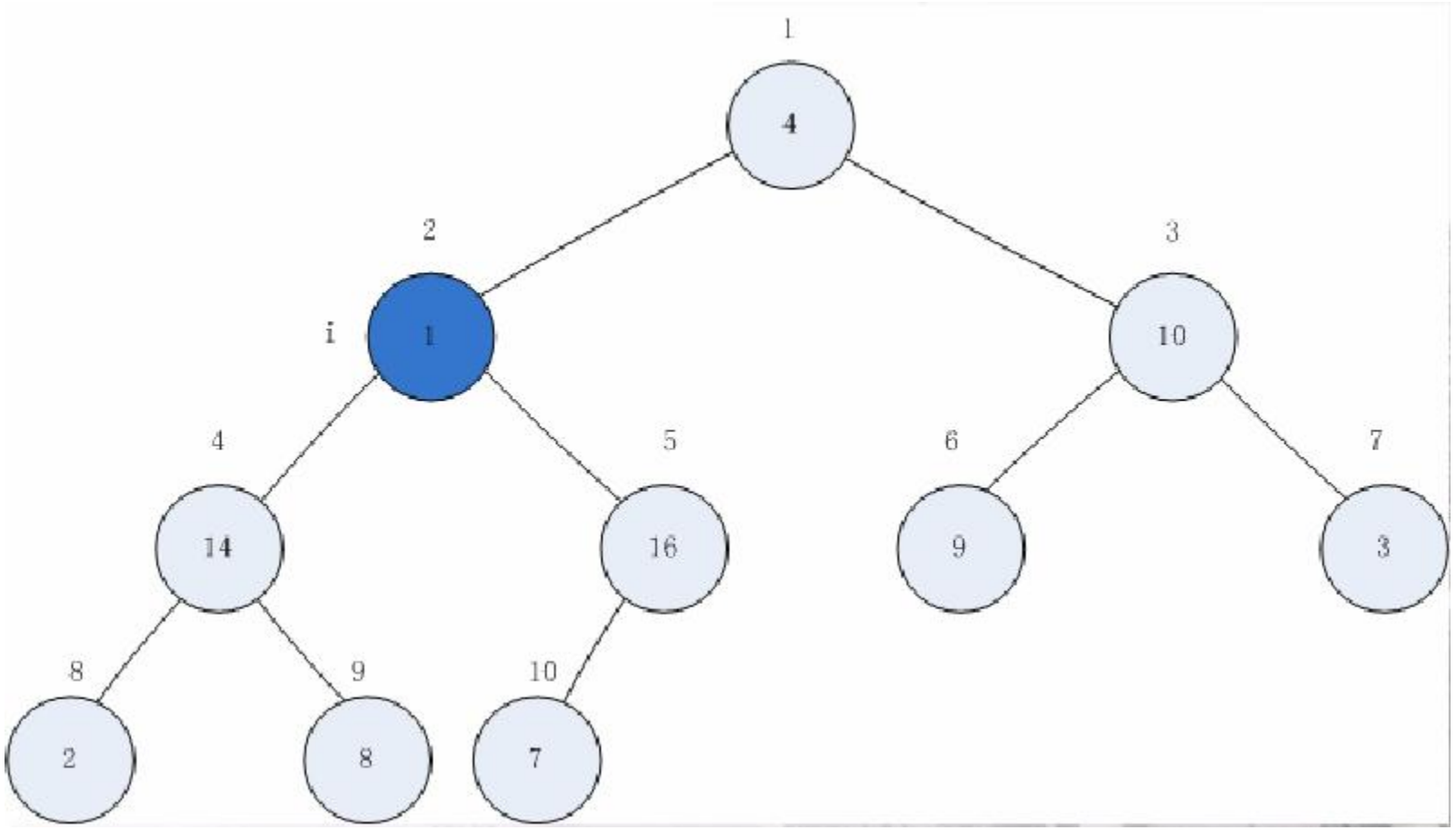
建堆过程



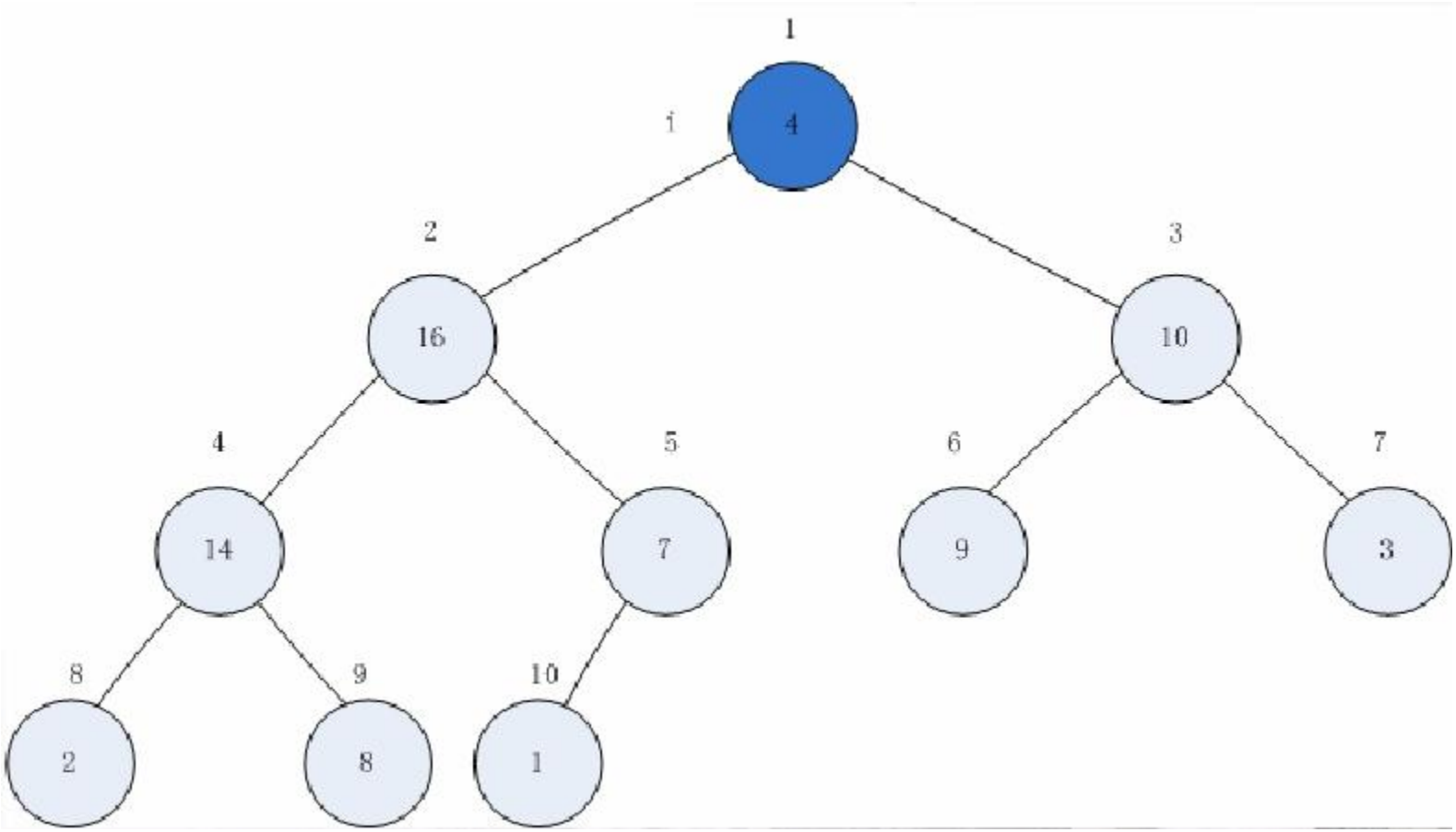
建堆过程



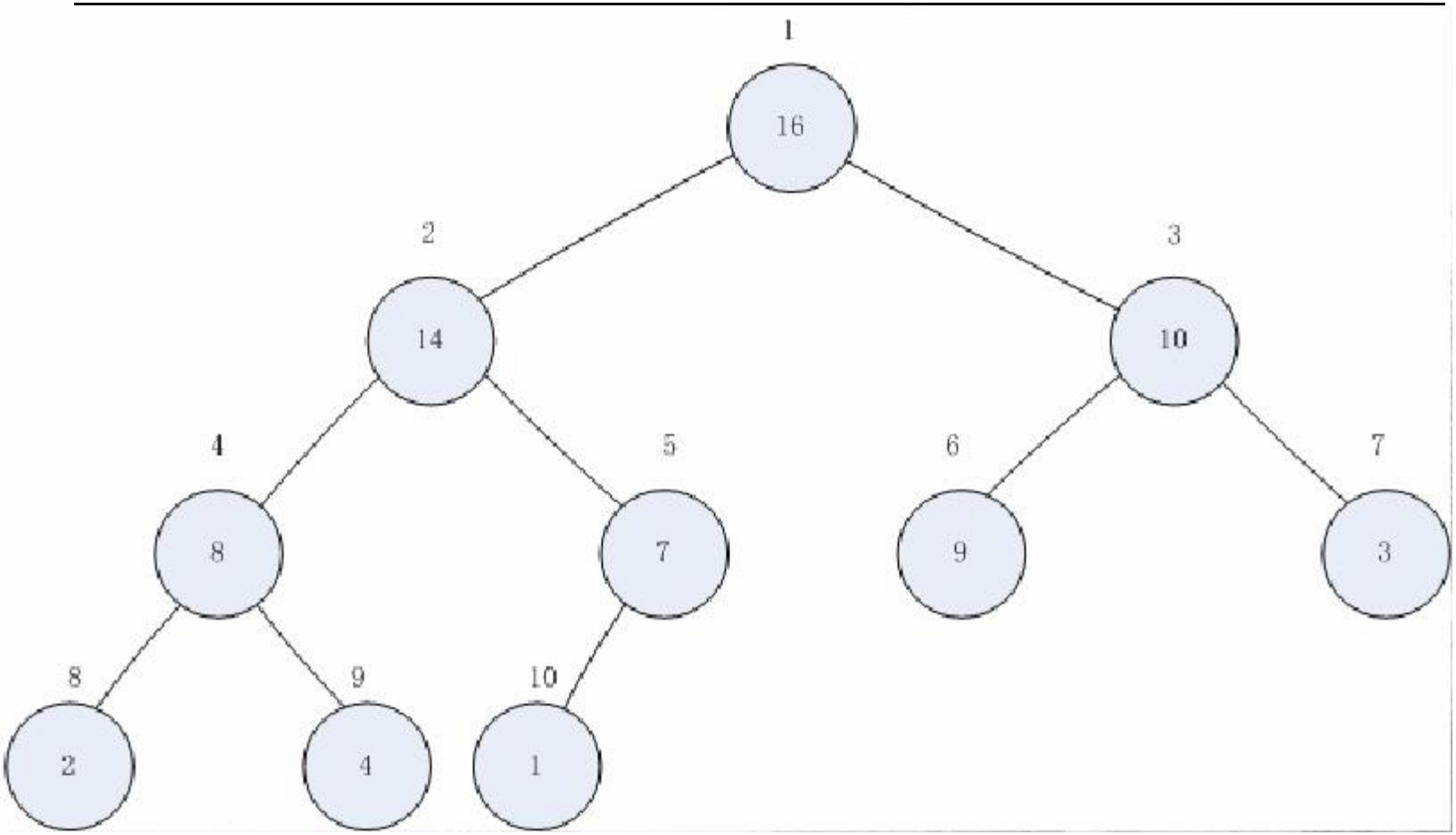
建堆过程



建堆过程



建堆过程



BUILD-MAX-HEAP上界

- 我们可以这样来计算BUILD-MAX-HEAP运行时间的一个简单上界：每次调用MAX-HEAPFY的时间为O(lgn)，共有O(n)次调用，故运行时间为O(nlgn)。这个界尽管是对的，但是从渐近意义上讲不够紧确。

BUILD-MAX-HEAP精确上界

- MAX-HEAPIFY作用在高度为h的结点上的时间为O(h)，我们可以将BUILD-MAX-HEAP的代价表达为上确界

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right)$$

将x=1/2代入几何级数的微分来计算(A.8)，则

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1-1/2)^2} = 2$$

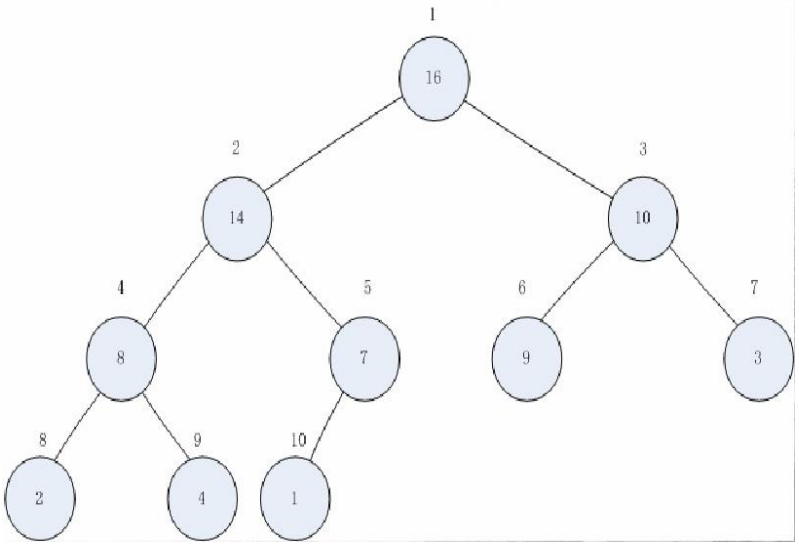
于是，BUILD-MAX-HEAP的运行时间的界为：

$$O\left(n \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h} \right) = O(n)$$

6.4 堆排序算法

- 开始时，堆排序算法先后用BUILD-MAX-HEAP将输入数组A[1..n]构造成一个最大堆，因为数组中最大元素在A[1]，可以通过把它与A[n]互换来达到最终正确的位置。

- 接下来，如果从堆中去掉结点n，可以很容易地将A[1..n-1]建成最大堆。原来根的子女仍是最大堆，而新的根元素可能违背了最大堆性质。这时调用MAX-HEAPIFY(A,1)就可以保持这一性质，在A[1..(n-1)]中构建出最大堆。



6.4 堆排序算法

- HEAPSORT(A)

  1 BUILD-MAX-HEAP(A)

  2 for i ←length[A] downto 2

  3 　　do exchange A[1]↔A[i]

  4 　　heap-size[A]←heap-size[A]-1

  5 　　MAX-HEAPIFY(A,1)

- HEAPSORT过程的时间代价为O(nlgn)，其中调用BUILD-MAX-HEAP的时间为O(n)，n次MAX-HEAPIFY调用中每一次的时间代价为O(lgn)。

6.4 优先级队列

- 堆排序的应用：（最大/最小）优先级队列
  - 作业调度：找出优先级最高的、插入作业、调整优先级
- 优先级队列是一种用来维护由一组元素构成的集合S的数据结构，这一组元素中的每一个都有一个关键字key。一个最大优先级队列支持以下操作：

INSERT(S,x)：把元素x插入集合S

MAXIMUM(S)：返回S中具有最大关键字的元素

EXTRACT-MAX(S)：去掉并返回S中的具有最大关键字的元素

INCREASE-KEY(S,x,k)：将元素x的关键字的值增加到k，这里k值不能小于x的原关键字的值。

优先级队列的操作

- 程序HEAP-MAXIMUM用$\Theta(1)$时间实现了MAXIMUM

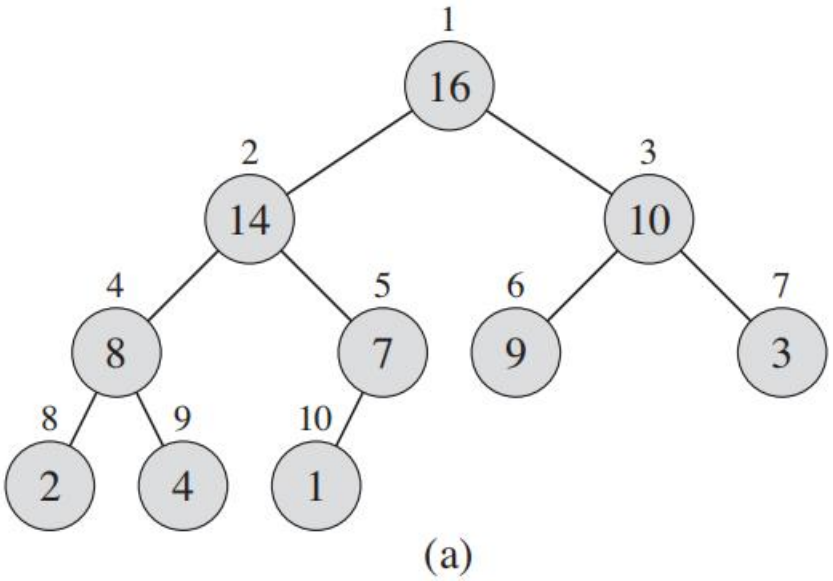
```
HEAP-MAXIMUM(A)
    return A[1]
```

- 程序HEAP-EXTRACT-MAX实现了EXTRACT-MAX

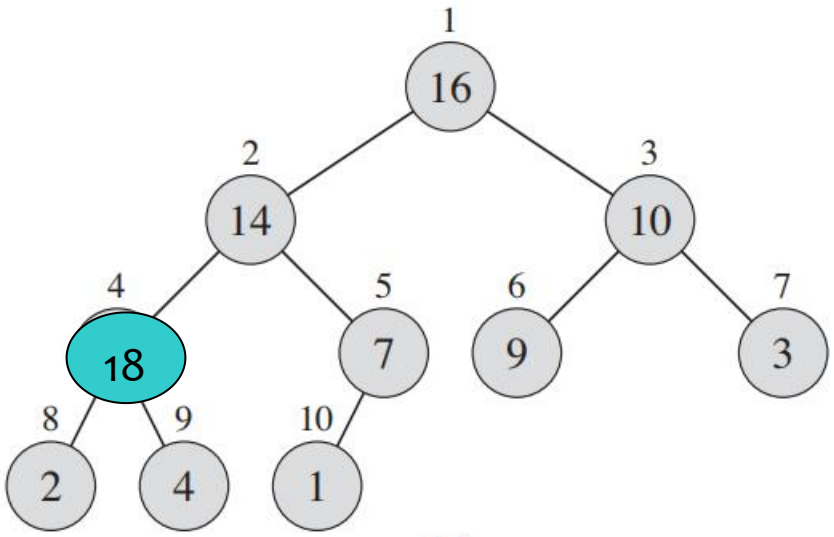
```
HEAP-EXTRACT-MAX(A)
if heap-size[A]<1
    then error ”heap underflow”
max←A[1]
A[1]←A[heap-size[A]]
heap-size[A]←heap-size[A] – 1
MAX-HEAPIFY(A,1)
return max
```

优先级队列的操作

程序HEAP-INCREASE-KEY实现了INCREASE-KEY操作。在优先级队列中关键字值需要增加的元素由对应数组的下标i来标识。



(a)



(b)

课内练习6.2

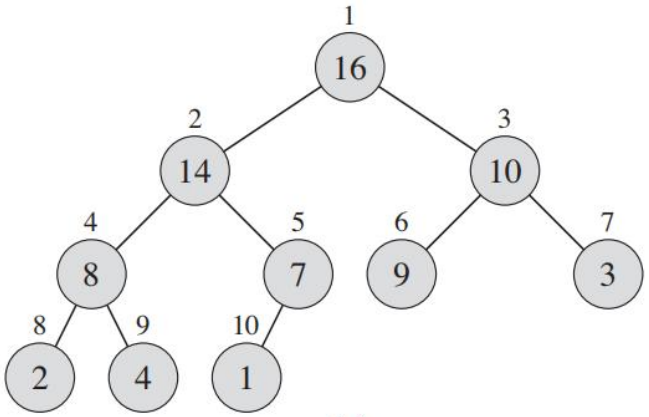
图（a)是一个最大堆，当节点8的关键字值增加到18时，如图(b)所示，请问答下列问题:

1、图(b)中以节点4为根的子树是否满足最大堆的定义?

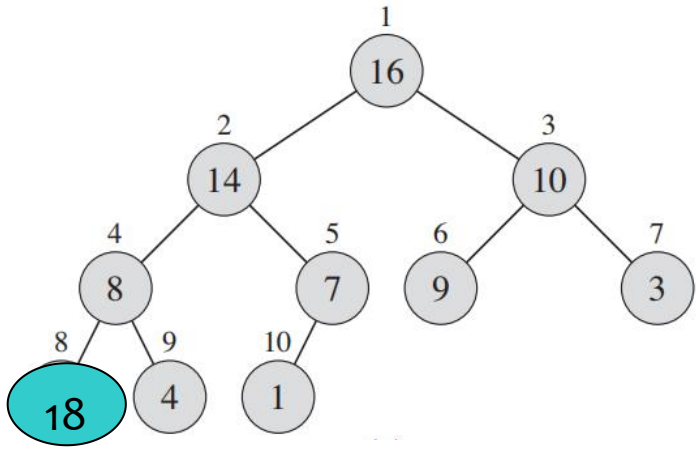
2、是否可以利用MAX-HEAPIFY对以节点4为根的子树进行调整，使它满足最大堆的定义。

3、以节点4为根的子树调整成为最大堆后，以节点2为根的子树是否满足最大堆的定义？如果不满足，利用问题2中的方法对其进行调整。

4、以此类推，写出HEAP-INCREASE-KEY(A,i,key)的算法描述，其中A是最大堆，i表示节点i，key是节点i更新后的关键字值（key>=节点i原来的关键字值）



(a)

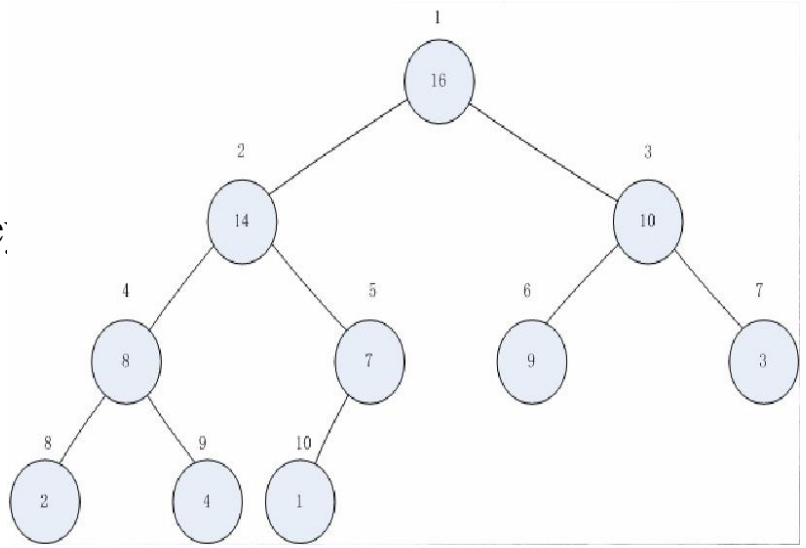


(b)

优先级队列的操作

- 程序HEAP-INCREASE-KEY实现了INCREASE-KEY操作。在优先级队列中关键字值需要增加的元素由对应数组的下标i来标识。该过程首先将元素A[i]的关键字值更新为新的值。由于增大A[i]的关键字可能会违反最大堆性质，因此在从本结点往根移动的路径上，为新增大的关键字寻找合适的位置。在移动的过程中，此元素不断地和其父母相比，如果此元素的关键字较大，则交换他们的关键字且继续移动。当元素的关键字小于其父母时，最大堆性质成立，程序终止

```
HEAP-INCREASE-KEY(A,i,key)
1 if key<A[i]
2     then error “new key is smaller than current ke
3 A[i]←key
4 while i > 1 and A[PARENT(i)]<A[i]
5     do exchange A[i]←→A[PARENT(i)]
6             i←PARENT(i)
```



优先级队列的操作

- 程序MAX-HEAP-INSERT实现了INSERT操作

  MAX-HEAP-INSERT(A,key)

  1 heap-size[A]←heap-size[A]+1

  2 A[heap-size[A]]←-∞

  3 HEAP-INCREASE-KEY(A, heap-size[A], key)